Лабораторная работа 15. Нагрузочный протокол, масштабирование и обзор LLMOps

Версия: 2026–2027 учебный год

Формат: команда 2–5 человек с индивидуальной сдачей

Продолжительность: 2 академических часа и самостоятельная работа

Предшествующая работа: ЛР14 «Документация, безопасность и Responsible ML»

1. Цель

Провести воспроизводимый steady/burst/recovery эксперимент, проверить SLO, локализовать bottleneck, сравнить capacity/cost варианты и оформить решение. Разобрать Kubernetes и LLMOps на обзорном уровне.

2. Результаты обучения

Каждый студент должен уметь:

• сформулировать workload model и условия воспроизводимости;

• получить и проверить сырые latency/status;

• рассчитать throughput, error rate, p50, p95 и max;

• объяснить minimum sample и coordinated omission;

• сопоставить latency с CPU, memory и saturation;

• провести recovery drill без разрушительных действий;

• обосновать keep/optimize/scale и оценку стоимости;

• объяснить HPA, probes, requests/limits и базовый LLMOps eval.

3. Командный результат

Команда предоставляет tag `lr15-v1`:

1. `docs/capacity-plan.md` с workload и SLO;
2. три повторных raw-профиля steady/burst;
3. recovery evidence с timestamps;
4. `capacity/load_policy.json` и итоговый capacity report;
5. графики/метрики ресурсов или обоснованное описание ограничения среды;
6. решение с bottleneck, альтернативой, стоимостью и rollback trigger;
7. review Kubernetes manifests и LLMOps eval cases;
8. тесты, CI и индивидуальные evidence cards.

4. Обязательный и обзорный контуры

Обязательны Docker Compose/API, Python load profile и capacity audit. Kubernetes cluster, metrics-server, внешний LLM API и платные вызовы не требуются. Manifests и eval cases проверяются как код и защищаются устно.

5. Подготовка

```
python -m pip install -e ".[dev]"
docker compose up --build --detach
curl http://127.0.0.1:8080/readyz
```

Зафиксируйте commit SHA, image ref, модель, CPU/RAM машины и отсутствие посторонней нагрузки.

6. Capacity policy

`capacity/load_policy.json` задаёт required scenarios, minimum requests, error-rate/p95 limits, recovery time, baseline/max replicas и учебную стоимость. Изменение policy после результата без отдельного обоснования считается подгонкой.

7. Warm-up

Скрипт выполняет пять запросов до измерения. Warm-up не смешивается со steady. Если модель загружается лениво или использует cache, увеличьте warm-up и зафиксируйте правило до сравнения вариантов.

8. Личный базовый запуск

```
python scripts/load_profile.py \
  --base-url http://127.0.0.1:8080 \
  --student-id student-01 \
  --requests-per-stage 40 \
  --steady-concurrency 2 \
  --burst-concurrency 10 \
  --output reports/capacity/lr15-student-01-run1-raw.json
```

Повторите три раза с теми же параметрами и уникальными output names.

9. Workload model

Учебный генератор использует закрытую модель с ThreadPoolExecutor. Укажите число workers, запросов, payload, timeout и среду. На защите объясните, почему медленный ответ уменьшает фактический arrival rate и как это связано с coordinated omission.

10. Сырые данные

Raw JSON хранит одну latency на запрос, successes/errors, wall duration и concurrency. Нельзя оставлять только скриншот terminal или готовый p95: преподаватель должен пересчитать percentile.

11. Capacity audit

```
python -m mlops_flight_delay.capacity \
  --results reports/capacity/lr15-student-01-run1-raw.json \
  --policy capacity/load_policy.json \
  --output reports/capacity/lr15-student-01-run1.json
```

`--enforce` разрешён для утверждённого release gate; экспериментальный fail сохраняется как evidence.

12. Percentiles

Проект использует nearest-rank: `ceil(q*n)`. Сравните p50, p95 и max. Покажите одну строку ручного расчёта или тест. Не округляйте входы до вычисления percentile.

13. Throughput

Throughput равен requests / wall duration. Он считается вместе с latency: рост throughput ценой timeouts не является улучшением. Укажите фактическую concurrency и проверьте приблизительное соответствие закону Литтла.

14. Error classification

Отделите timeout/connection/5xx от намеренных 4xx. В обязательном load profile payload валиден. Если добавлен negative scenario, его 422 хранится в отдельном результате и не смешивается с capacity error rate.

15. Наблюдение ресурсов

Во время steady и burst сохраните CPU, memory и request rate из Prometheus/Grafana либо `docker stats`. Укажите timestamps, чтобы сопоставить ресурс и p95. Скриншот без периода и параметров нагрузки недостаточен.

16. Bottleneck hypothesis

Выберите один наблюдаемый bottleneck или докажите, что он не достигнут. Рассмотрите CPU, memory, client, I/O, worker/connection limit и загрузку модели. Гипотеза должна предсказывать эффект конкретного изменения.

17. Контролируемый эксперимент

Измените один фактор: concurrency, workers, CPU limit или replicas в локальном Compose-эквиваленте. Не меняйте одновременно код модели и инфраструктуру. Повторите тот же профиль и сравните таблицу.

18. Recovery drill

В согласованное окно выполните безопасный `docker compose restart api`. Измерьте время от команды до трёх последовательных 200 на `/readyz`; сохраните timestamps и ошибки. Не удаляйте volumes и не используйте реальные production-сервисы.

19. Критерий устойчивой готовности

Один ответ 200 не завершает recovery. Требуются три последовательные успешные readiness checks. Если сервис флапает, зафиксируйте последовательность, logs и причину вместо усреднения.

20. Решение о capacity

`pass` предлагает `keep_baseline`. `review` предлагает `investigate_then_scale`, а не автоматическое увеличение replicas. Команда принимает итоговое keep/optimize/scale/limit/redesign с владельцем.

21. Стоимость

Сравните baseline и candidate в условных единицах replica-hour. Укажите, что не учтены registry, load balancer, storage, egress и сопровождение. Не выдавайте учебную оценку за тариф провайдера.

22. Kubernetes Deployment review

Проверьте immutable image, rolling strategy, non-root, read-only filesystem, requests/limits, startup/readiness/liveness и termination grace. Каждая probe должна иметь отдельную семантику.

23. HPA review

`k8s/hpa.yaml` использует `autoscaling/v2`, 2–6 replicas, CPU target и scale-down stabilization. Объясните необходимость resource requests и metrics pipeline. CPU HPA не гарантирует p95 и может не помочь при I/O bottleneck.

24. PDB review

`k8s/pdb.yaml` ограничивает добровольные disruptions до `minAvailable: 1`. Объясните, какие crash и ошибки он не предотвращает. Kubernetes manifests не применяются к чужому cluster.

25. LLMOps mapping

Сопоставьте классический артефакт с LLMOps:

MLOps	LLMOps
model/data version	provider/model, prompt, retrieval corpus
prediction log	trace шагов и tool calls
test set	eval cases и critical slices
metric gate	deterministic/model/human scorers
latency/cost	TTFT, total latency, tokens и стоимость ответа
input security	injection, data leakage и tool boundaries

26. Eval cases

`llmops/eval_cases.json` содержит format, refusal, injection и grounding cases. Каждый участник добавляет или рецензирует один безопасный case и указывает тип scorer. Реальный API не вызывается, секрет не создаётся.

27. MLflow GenAI обзор

Покажите в документации назначение tracing и `mlflow.genai.evaluate()`. Не смешивайте его с `mlflow.models.evaluate()` классического ML. Укажите, какие поля trace нельзя сохранять безусловно.

28. Автоматические тесты

Обязательны проверки:

1. nearest-rank percentile детерминирован;
2. baseline проходит policy;
3. высокий burst p95 приводит к `review`;
4. отсутствующий scenario отклоняется;
5. decision не обещает автоматическое масштабирование;
6. JSON/YAML parseable и версии согласованы.

29. CI

CI анализирует синтетический sample и сохраняет `reports/capacity/lr15-ci.json`. Live load test не запускается на shared runner: его шумный результат не является стабильным unit gate. Собственные raw прогоны прикладываются как artifacts к PR.

30. Командные роли

Для команды 2–5 человек:

• владелец workload/protocol и генератора;

• владелец metrics/SLO и capacity audit;

• владелец resource observation и bottleneck experiment;

• владелец recovery/Kubernetes review;

• владелец cost decision и LLMOps eval review.

В паре роли совмещаются; координатор без измерения, review или технического артефакта не засчитывается.

31. Индивидуальная сдача участника команды

Командный результат: tag `lr15-v1`, протокол, raw runs, capacity report, recovery и decision record.

Каждый заполняет `docs/evidence/lr15-evidence-card.md`; совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР15, tag и commit SHA
Роль	роль и отличие от ЛР14
Задача	собственная issue с критерием приёмки
Личный артефакт	профиль, тест, SLO, recovery, manifest review или eval case
Среда	CPU/RAM, image/model version, команда запуска
Evidence	requests, concurrency, throughput, p50/p95/max, errors, recovery, resources
Решение	bottleneck, альтернатива, cost/risk и rollback trigger
Review	содержательное замечание и принятое решение
Использование ИИ	инструмент и самостоятельно проверенные утверждения

32. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Участнику назначена собственная issue.
3. Есть личный raw output либо иной проверяемый технический артефакт.
4. PR проверен другим участником.
5. Параметры и среда воспроизводимы.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент объясняет ограничение метода.

33. Индивидуальная защита

За 3–4 минуты участник:

1. показывает свой артефакт в tag;
2. пересчитывает p95 или throughput;
3. интерпретирует resource evidence;
4. меняет concurrency/SLO по заданию;
5. объясняет bottleneck и альтернативу scale;
6. читает один фрагмент HPA/probe;
7. классифицирует LLMOps eval case и его риск.

34. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, отдельный PR, новый raw run и evidence card.

История первой попытки сохраняется.

Стоп-факторы:

• изображение графика заменяет raw данные;

• сравниваются разные версии/среды без указания;

• 4xx смешаны с server capacity errors;

• p95 рассчитан без фиксированного метода;

• один аномальный run объявлен доказательством;

• replicas увеличены без bottleneck evidence;

• стоимость представлена как точный тариф;

• Kubernetes cluster или внешний LLM ошибочно сделан обязательным;

• секрет/API key попал в Git;

• общий run выдан за личный;

• студент не объясняет свой вывод.

35. Критерии оценивания

Критерий	Баллы	Полное выполнение
Протокол и воспроизводимость	2	workload, версии, среда и raw data зафиксированы
Метрики и SLO	2	throughput, errors, p50/p95/max рассчитаны корректно
Bottleneck и recovery	2	ресурсы сопоставлены, recovery доказан timestamps
Capacity/cost, Kubernetes, LLMOps	2	решение и ограничения объяснены без магии
Индивидуальный вклад и защита	2	личный артефакт, review и диагностика воспроизводимы

Максимум: 10 баллов. Порог зачёта — 6 баллов при отсутствии стоп-факторов.

36. Чек-лист

• [ ] Команда 2–5 человек, роли ротированы.

• [ ] Commit, image, model и policy versions записаны.

• [ ] Warm-up отделён от измерения.

• [ ] Есть три повторных steady/burst runs.

• [ ] Сырые latency сохранены.

• [ ] p50/p95/max и throughput пересчитываются.

• [ ] Ошибки классифицированы.

• [ ] Ресурсы связаны с тем же периодом.

• [ ] Recovery требует трёх readiness successes.

• [ ] Bottleneck подтверждён либо ограничение признано.

• [ ] Cost assumptions явны.

• [ ] Kubernetes остаётся обзором/демо.

• [ ] LLMOps eval не требует секрета.

• [ ] CI и индивидуальные cards готовы.

• [ ] Итог отмечен tag `lr15-v1`.

37. Связь с ЛР16

В ЛР16 выбранный capacity decision, recovery evidence, model/data versions, image ref и governance status войдут в единый release manifest. Команда проведёт итоговую end-to-end демонстрацию и индивидуальную защиту сквозного проекта.

38. Официальные источники

• Kubernetes HPA: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

• Kubernetes resources: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

• Kubernetes probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/

• MLflow GenAI tracing: https://mlflow.org/docs/latest/genai/getting-started/tracing

• MLflow GenAI evaluation API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.genai.html